Latest updates: https://dl.acm.org/doi/10.1145/3671016.3674819

RESEARCH-ARTICLE

RepoMinCoder: Improving Repository-Level Code Generation Based on Information Loss Screening

YIFAN LI, Sun Yat-Sen University, Guangzhou, Guangdong, China

ENSHENG SHI, Xi'an Jiaotong University, Xi'an, Shaanxi, China

DEWU ZHENG, Sun Yat-Sen University, Guangzhou, Guangdong, China

KEFENG DUAN, Sun Yat-Sen University, Guangzhou, Guangdong, China

JIACHI CHEN, Sun Yat-Sen University, Guangzhou, Guangdong, China

YANLIN WANG, Sun Yat-Sen University, Guangzhou, Guangdong, China

Open Access Support provided by:

Sun Yat-Sen University

Xi'an Jiaotong University

PDF Download
3671016.3674819.pdf
31 March 2026
Total Citations: 4
Total Downloads: 411

Published: 24 July 2024

Citation in BibTeX format

Internetware 2024: 15th Asia-Pacific Symposium on Internetware
July 24 - 26, 2024
Macau, China

Conference Sponsors:
SIGSOFT

Internetware '24: Proceedings of the 15th Asia-Pacific Symposium on Internetware (July 2024)
https://doi.org/10.1145/3671016.3674819
ISBN: 9798400707056

RepoMinCoder: Improving Repository-Level Code Generation Based on Information Loss Screening

Yifan Li*
Sun Yat-sen University
Zhuhai,China
liyf283@126.com

Ensheng Shi*
Xi'an Jiaotong University
Xi'an,China
s1530129650@stu.xjtu.edu.cn

Dewu Zheng
Sun Yat-sen University
Zhuhai, China
zhengdw5@mail2.sysu.edu.cn

Kefeng Duan
Sun Yat-sen University
Zhuhai,China
duankf@mail2.sysu.edu.cn

Jiachi Chen
Sun Yat-sen University
Zhuhai,China
chenjch86@mail.sysu.edu.cn

Yanlin Wang†
Sun Yat-sen University
Zhuhai,China
wangylin36@mail.sysu.edu.cn

ABSTRACT

Repository-level code generation task involves generating code at a specified location based on unfinished code with repository context. Existing research mainly rely on retrieval-augmented generation methods to complete code. Existing work mainly investigates on improving retrieval results based on the unfinished code, but rarely pays attention to the information loss in the prompt encoding process. In this paper, we propose RepoMinCoder, a novel repository-level code generation framework that adds another round of screening and ranking based on information loss, building upon the canonical retrieval-augmented generation method. Extensive experimental results demonstrate that RepoMinCoder consistently outperforms state-of-the-art methods on public benchmark RepoEval, achieving 3.3% EM and 2.1% ES improvement over previous methods. Moreover, we conduct additional experiments to study the effect of various factors in the existing code generation pipeline, including the number of retrieval candidates, the slicing strategy of the retrieval database, and different prompting strategies.

KEYWORDS

Code Generation, Large Language Model, Screening and Ranking

ACM Reference Format:
Yifan Li, Ensheng Shi, Dewu Zheng, Kefeng Duan, Jiachi Chen, and Yanlin Wang. 2024. RepoMinCoder: Improving Repository-Level Code Generation Based on Information Loss Screening. In *15th Asia-Pacific Symposium on Internetware (Internetware 2024), July 24–26, 2024, Macau, China.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3671016.3674819

*Equal contribution
†Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Internetware 2024, July 24–26, 2024, Macau, China
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0705-6/24/07
https://doi.org/10.1145/3671016.3674819

1 INTRODUCTION

The task of repository-level code completion is to generate expected code given the unfinished code with repository context [2, 21, 38, 39, 41]. It is more realistic and challenging than common code completion task because in a repository there are multiple code files and complex dependencies. The content to be generated is only a small part, and the variables and functions it depends on are often in different files. Providing this information to a large language model (LLM) can effectively improve performance in repository-level code completion tasks. Studies have shown that providing different prompts to a LLM has a significant impact on the generated results. Well-crafted, relevant prompts can enhance performance, while unrelated or poorly described prompts can hinder it. Additionally, excessively long prompts can reduce efficiency. Therefore, studying how to provide excellent prompts to large language models (LLMs) is crucial.

In the repository-level code completion task, there has been considerable research on leveraging information from associated repositories to generate prompts. Among these, the retrieval-augmented generation method is widely used [2, 21, 38, 39, 41]. However, there is still a lack of research on how to select and sort the retrieved snippets and whether the content in the snippets is entirely appropriate. Additionally, in the pursuit of further performance, the concept of iteration is introduced, but too many iterations may have negative impact on efficiency.

In this paper, we propose a new method RepoMinCoder(stands for RepoCoder and minimum description length principle). This method improves upon RepoCoder [41]. As shown in Figure 1, we have added an additional step to RepoCoder: an information loss-based selection and ranking method, MDL [37]. Unlike RepoCoder [41], which directly selects the top k most similar code snippets as prompts, RepoMinCoder takes into account information loss to determine the final selection and order of the code snippets used as prompts. Based on similarity, we select more code snippets than ultimately needed as candidates and then perform permutations. We calculate the information loss for different permutation orders and choose the one with the least loss as the prompt.

Through experiments, we find that the RepoMinCoder method improved the accuracy of code generation. We also investigate the impact of certain details within RepoMinCoder on the generation performance. Specifically, we explore the effect of the number of

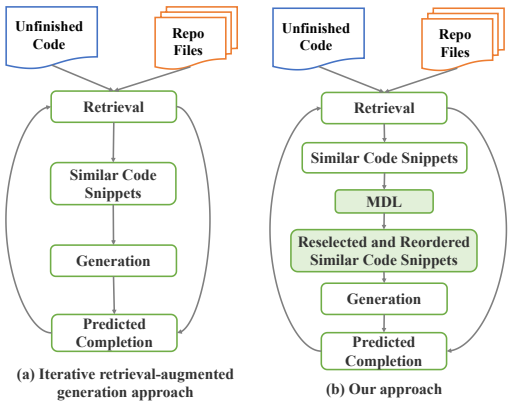

Figure 1: Iterative retrieval-augmented generation approach v.s. RepoMinCoder (ours).

candidate code snippets and the method of dividing code snippets on the final results. The experimental results indicate that more candidate code snippets do not necessarily lead to better outcomes. Additionally, for code division, the sliding window approach is proved to be superior.

In this study, our main contributions are as follows:

- We bring a new perspective based on information compression and loss to the repository-level code generation task, which considers the selection and order of prompt content.
- We investigate a new selection sorting method and propose a new high-performance retrieval-augmented generation method for repository-level code generation.
- We conduct extensive experiments to explore the effectiveness our proposed method. Our findings offer new insights for further research in this area.

2 RELATED WORK

2.1 Large language models for code generation

The emergence of LLMs has garnered significant attention. Many scholars have dedicated themselves to related research, exploring various aspects such as prompts, fine-tuning, principles, and construction of test datasets to study, enhance the performance of, and broaden the applications of LLMs. Following the development of LLMs for natural language, LLMs for code generation have emerged.

On the basis of the GPT model, Chen et al.[3] developed Codex, a large language model for code generation. They used public code from GitHub for fine-tuning and proposed the HumanEval test set to evaluate the model's performance. Although the model has been able to solve some problems, it still struggles with docstrings that need to describe long operation chains and variable binding operations. Nijkamp et al. [24] used three datasets—ThePile, BigQuery, and BigPython—to train the CodeGen series of LLMs. These models include CodeGen-NL for natural language, CodeGen-MULTI for multiple programming languages, and CodeGen-MONO for Python. While these LLMs demonstrate excellent performance, the resulting programs still have limitations and can pose security risks. Roziere et al. [28] released Code Llama, which is based on Llama 2, a large code generation model trained on a substantial amount of data. Code Llama offers improved performance and provides various applications and parameter size options. Li et al. [19] obtained training data from licensed GitHub libraries and processed it by de-weighting, weighting, ensuring uniform format, and removing personal information. They train StarCoder on these data and demonstrated the effectiveness of StarCoder.

2.2 Repository-level code generation

After the initial application of simple code generation techniques, scholars have begun to delve into the utilization of LLMs for repository-level code generation. Generating code at the repository level is significantly more complex than generating code for individual functions. The challenges include, but are not limited to, referencing other files within the same project, accessing private APIs, and managing global modifications caused by version iterations.

Zhang et al. [41] introduced the RepoCoder framework along with a suite of test sets called RepoEval. The framework is structured into two main components: retrieval and generation, employing an iterative approach where previously generated content informs subsequent retrieval. RepoEval comprises three test sets at varying levels of complexity: line, API invocation, and function. The RepoCoder framework was evaluated using RepoEval and demonstrated favorable results.

Zan et al. [38, 39] found that private library API calls are more prone to errors, and proposed two groups of frameworks: APIFinder and APICoder, APIRetriever and APICoder to solve this problem. APIFinder uses vector retrieval technology and allows users to participate in the retrieval process. APIRetriever is a dense search system with a user-friendly interactive approach to use. The APIRetriever framework retrieves potentially useful APIs by comparing the problem context with the contents of the API document. The retrieved information is then transferred to the large language model for code generation along with the problem context.

Liu et al. [21] studied the problem of code generation with natural language and third-party libraries and proposed the CodeGen4Libs framework. This framework divides the task of whole generation into Import generation and code generation. First, the third-party library related to the problem context is retrieved and used to assist the large language model in completing the code generation of the Import section. Then, retrieval is conducted again to find code snippets similar to the context of the problem. Finally, these snippets, along with the problem context and the generated Import sections, are used together as prompts for the large language model to complete the code generation.

Bairi et al. [2] proposed the CodePlan framework, considering that local changes in a project may have an impact on the global situation. This framework builds a dependency table based on the relationships between functions and variables in a project. When a change occurs somewhere in the project, it is detected according to the dependency graph to find relevant parts that need to be updated. The process is iterative, and newly detected parts that need to be updated after modification will continue to be detected.

2.3 Prompting methods for code generation

When using a large language model to generate code, in addition to the problem itself, some relevant information is often provided as a prompt to improve the accuracy of the generated results.

One way of doing this is through the Chain of Thought (CoT) approach. CoT was first proposed by Wei et al. [34] and applied to

mathematical reasoning problems in natural language, achieving great results. Since then, the chain of thought approach has also begun to be tried in the field of code generation. Li et al. [17] proposed a structured thought chain for code to guide LLMs to think step by step. First, the input and output content, along with the branch structure, loop structure, and sequence structure of the process, are deduced, and then they are regenerated into code. Jiang et al. [15] proposed a method that first lets the large language model deduce a plan for the code to be generated, and then gives the plan as a prompt to the large language model for code generation.

Another common approach is to retrieve information related to the problem within a certain range as a prompt. In terms of code generation, AceCoder, proposed by Li et al. [18], uses this technique. This framework retrieves code from a corpus that is similar to the problem and prepares test cases for that code, presenting them together as hints to the larger model. Additionally, RepoCoder [41], APIRetriever [39], and CodeGen4Libs [21] also use retrieval methods.

In addition to increasing the prompt content to improve the accuracy of content generation, there is also work from an efficiency perspective aimed at reducing the length of the prompt. Zhang et al. [42] studied the CodeBERT [6] model and proposed a method to optimize and delete input content from LLMs based on attention weights, successfully reducing computing costs. However, their paper focuses on code search and code summarization rather than code generation. There is still a lack of efficiency-focused work on the code generation side.

3 PROPOSED APPROACH

3.1 Overview

In this study, the prompt for the repository-level code completion task consists of two parts. One part is the task itself, including the preceding code snippet of the vacancy. The other part comprises some useful code snippets obtained through the retrieval-augmented generation method. Ideally, it is best to provide all relevant information when solving the repository-level code completion problem, but the prompt length limitation of LLMs makes it impossible to use the entire project code file as a prompt. Moreover, even if the length is not limited, not all code is relevant to the task itself, and the irrelevant parts could interfere with the large language model's predictions and affect the results.

Because of these constraints, it becomes important to select as many useful parts as possible. To filter effectively, we use the retrieval-augmented generation method. According to relevant literature, the retrieval-augmented generation method is widely used in code generation tasks based on LLMs and in repository-level code completion tasks. It achieve good results.

The retrieval range in this study includes all the code files of the project to be completed. Using the code from the same project as the prompt can effectively ensure the relevance of the code and the consistency of the code style, as well as reduce factual errors. The retrieval method we use is similarity comparison. We split the code in the project files into snippets of a fixed number of lines, ensuring that the code snippets are similar in length to minimize the possibility of them being too long or too short. This also ensures the integrity of the code snippets to a certain extent.

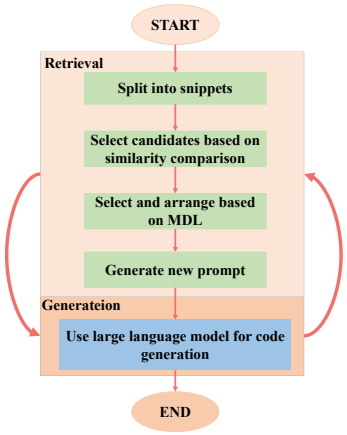

Figure 2: The pipeline of RepoMinCoder.

The preceding code snippet of the vacancy is also truncated to minimize the impact of snippet length. These code snippets are then compared to the preceding code snippet of the vacancy. Based on the similarity scores, several code snippets with higher similarity are selected as candidates for the second round of screening.

In order to further improve the accuracy of the large language model generation results, we employ an iterative approach. This method leverages the initial large language model generation results and builds upon them. When it comes to retrieval, the validity of the snippets retrieved using the question itself is limited. Often, similar questions do not necessarily result in similar answers. However, even if the answer is not correct, it tends to be close to the correct answer or at least have some similarity. The same principle applies to LLMs for code generation. Although the answer may be incorrect, it is still moving in the right direction and thus has some value that can be utilized.

For the first retrieval, we only use the preceding code snippet of the vacancy to compare with the code snippets. For the second retrieval, we use both the preceding code snippet of the vacancy and the previous generation result to compare with the code snippets. However, when using a large language model for prediction, the previous prediction results are not provided to the model.

During the first retrieval process, we add all the selected code snippets backward a few lines and delete the first few lines. This is because when performing the initial similarity comparison, only the preceding code snippet of the vacancy is used as a reference. The code snippets obtained through this process are only similar to the preceding code, which actually represents the similarity of the problems. Therefore, to generate the following code, what we actually need is the following code of the similar snippets.

Based on the iterative retrieval-augmented generation method, we further refine the process by integrating the Minimum Description Length (MDL) method [37] to conduct an additional screening and ranking of the candidate code snippets, which constitutes the second round of screening mentioned earlier. During this operation, the large language model converts the prompted string into a matrix, potentially leading to information loss. The MDL method is designed to minimize this loss.

Initially, we organize and combine the highly similar candidate code snippets that have been selected. For each combination, we calculate the information loss incurred during the conversion process,

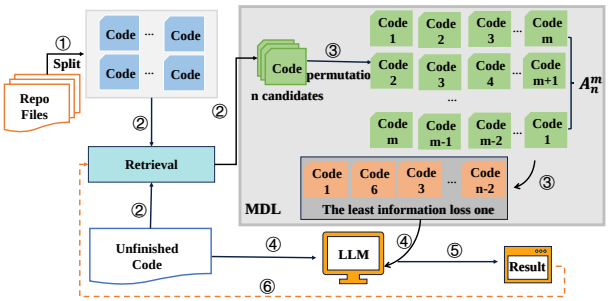

Figure 3: RepoMinCoder Architecture.

transforming the abstract loss into a measurable quantity. Subsequently, we compare and select the combination with the least information loss as the final prompt for the large language model. This method not only completes the selection of code snippets but also sorts them.

In addition, we explore the selection of specific content. The content contained in an associated repository is very rich, including unique elements such as comments and function signatures. We investigated whether the comments presented are suitable for use in LLMs and whether using only function signatures as prompts might be more effective. We also examined whether there are other ways to simplify prompts to improve efficiency without significantly affecting performance.

To this end, we have made several attempts. Besides starting with function signatures and comments, we also look into DietCode [42] and try to apply its principles to the repository-level code completion task. These explorations aim to determine if certain types of content could be more effective as prompts and if techniques from DietCode could help streamline the process while maintaining or improving the performance of the large language model in code generation tasks.

3.2 Retrieval and generation

In the retrieval process, we need to use a retrieval and selection model, here we call it R. The specific content about this retrieval model will be introduced later. The range of the retrieval, mentioned earlier, is the code file for the entire project. The length of the code in these code files varies. But most of them are too long. Selecting directly by file granularity is obviously not practical. To solve this, we will split the code from the original project into snippets $c_i (i = 1, 2, ...)$ that can be used for comparison and as prompts. For the collection of these code snippets, we call it C_{repo}, $C_{repo} = \{c_1, c_2, ...\}$. After that, we will give the preceding code snippet of the vacancy and the collection of code snippets to the retrieval model to retrieve the code snippets that are more suitable for the code generation task. The whole retrieval process can be expressed as $C_{return} = R(C_{repo}, X)$. Where X represents the preceding code of the vacancy. C_{return} represents the retrieved snippets of code that will be selected as a prompt.

After obtaining the retrieved code snippet C_{return}, we use the fixed format T to combine it with X that the preceding code snippet of the vacancy to form a new prompt $X^{'} = T(C_{return}, X)$.After that, we will give the new prompt $X^{'}$ to the large language model M for generation. We will get the result Y generated by the large language model. The process is described by the formula $Y = M(X^{'})$.

Next, we will use the generated result Y to assist the retrieval, $C^{'}{}_{return} = R(C_{repo}, X + Y)$. $C^{'}{}_{return}$ indicates the anew retrieved snippets. Again, we use the fixed format to form a new prompt, $X^{''} = T(C^{'}{}_{return}, X)$. And then, we use a large language model to generate, $Y^{'} = M(X^{''})$. The generation result $Y^{'}$ is the final result.

3.3 Candidates construction and similarity comparison

The content of project-level code is typically very extensive. To facilitate retrieval, the code must first be segmented. We divide the code into snippets based on a fixed number of lines. We set a window with a fixed number of rows and specify the number of rows that the window slides down at a time. Each swipe of the window generates a code snippet.

Initially, we considered partitioning the code based on function granularity. However, the performance of this method was not satisfactory and was worse than that of using a fixed number of lines. Therefore, we ultimately chose to partition the code using a fixed number of rows.

After dividing the code into snippets, we need to retrieve them based on similarity comparison. In this study, we use the Jaccard similarity method [14]. This method measures the similarity between two pieces of code by comparing the number of identical words between them. The similarity scores obtained range between zero and one, with scores closer to one indicating higher similarity. Since this method does not require converting strings to vectors, we do not need to use other models to process the strings. We simply divide the strings into words for comparison.

At this point, the code snippet has been evaluated using a similarity score. The next step is selection. First, the top-k method is used to select some snippets as candidate snippets based on their similarity scores. Then, the MDL method [37] is employed to determine which snippets are chosen from the candidate snippets and the order in which they are selected. We will discuss the MDL method [37] in more detail later.

3.4 Selection and sorting method

The MDL method [37] is a selection sorting algorithm based on the minimum description length (MDL) principle [8].

The minimum description length principle was proposed by Rissanen in his study of universal coding [8]. For a given set of instance data, to save storage space, a model is typically used to encode and compress it, and then the compressed data is saved. Simultaneously, the model itself is saved to properly recover the instance data later. Therefore, the length of data to be saved (in bits) is equal to the length of the instance data after encoding and compression plus the length of data required to save the model, which is called the total description length. The MDL principle requires selecting the model with the smallest total description length. According to this principle, the model that adheres to the MDL principle can minimize the information loss caused by compression.

The selection and sorting method we use is inspired by this idea. During the generation of a large language model, the prompts we provide, although initially strings, are converted into matrices. Information may be lost in this transformation process. Whether the information can be effectively transmitted to the large model affects

the generation outcome. Based on this, Wu et al. [37] proposed the MDL method, which applies the minimum description length principle to the ordering and selection of prompts for natural language problems using LLMs. We attempt to apply this method to code generation problems based on LLMs.

Next, we will introduce the formula derivation of the MDL method [37]. According to the principle of minimum description length, we can get the measurement formula as follows:

$$e^{'} = \arg\min_{e \in E} CL_{\theta}(trans|e, ori) + CL(\theta) \tag{1}$$

In the formula above, e represents a kind of combination and selection of code snippets. ori is the original prompt. $trans$represents the converted and compressed test sign. $CL_{\theta}(trans|e, ori)$ represents the code length of the test tag after conversion and compression. $CL(\theta)$ is the code length of the compressed model, which is generally small and often negligible.

As for the calculation of code length, according toShannon-Hufu coding, we can get the formula as follows on the basis of the original formula:

$$CL_{\theta}(trans|e, ori) = -log_2 p(trans|e, ori) \tag{2}$$

However, there is no way to use this formula. Because we can't calculate $p(trans|e, ori)$ exactly. This problem is transformed into an expectation calculation problem. The formula then becomes:

$$CL_{\theta}(trans|e, ori) = -E_{q(trans_i|Trans)} p(trans_i|e, ori) \tag{3}$$

$q(trans_i|Trans)$ represents the prior probability of occurrence of $trans_i$ in all signs $Trans$. However, since the prompt snippets chosen for different problems are different, calculating prior probabilities is of little use. Therefore, the calculation of the prior probability $q(trans_i|Trans)$ is simplified to $p(trans_i|e, ori)$.Finally, the formula is as follows:

$$e^{'} = \arg\min_{e \in E} -E_{p(trans_i|e,ori)} p(trans_i|e, ori) \tag{4}$$

By using the above formula to calculate and compare each combination separately, we can determine which one is the best.

In addition, there is the question of the quantity of combinations that needs to be mentioned. We cannot determine the best combination until we perform a computational comparison. Therefore, to choose the most suitable one, it is best to use the method of enumeration. We need to list all possible combinations and calculate them separately, considering different orders of the same snippets as distinct possibilities. For example, if there are five candidate snippets and we need four, the number of possible combinations is one hundred and twenty. When this method is applied to natural language problems, the cost of enumeration becomes very high or even unbearable due to the large number of candidate snippets and required snippets. Therefore, we have to take a random approach. Although not the best, it greatly reduces the computational consumption. For code generation tasks, this problem does not exist for the time being.

3.5 Prompt simplification

To increase efficiency, we consider reducing the content of the prompts. Fewer prompts mean less time needed for the large language model to generate responses. Most previous studies start from the whole code, whether comparing similarity or relevance, without paying attention to the specific content. However, there are differing opinions on whether function signatures, annotations, plain snippets of code, or other divided code segments hold the same value.

Initially, we consider selecting only the function signature as prompts. There are many studies on function signatures and repository code generation tasks that show the value of function signatures [22, 38, 39]. In our approach, we retrieve relevant code snippets based on their similarity. However, the prompt provided to the LLM is not the code itself but rather the function signature of the function contained in or related to the code. We utilize the retrieval-augmented generation method in this approach to provide additional prompts containing only function signatures.

Next, we address the issue of annotations. In the associated repository serving as the test set, both the code and annotations are human-completed. While the code must be syntactically compliant, there are no strict requirements for annotations. Many annotations are unreadable, potentially posing readability challenges for individuals. In scenarios where code is already provided, such annotations may become intrusive if used as prompts for a large language model. Therefore, we propose removing all annotations from the retrieved code snippets before feeding them to the large language model.

To achieve this, we start with the first line of each code snippet to determine if it contains a single-line or multi-line annotation. For single-line annotations, we delete the symbol and everything to the right. For multi-line annotations, we identify whether it is the beginning or end, record its position, find the corresponding symbol, and delete the entire annotation content. As a result, only processed code snippets without annotations are provided to the large language model as prompts.

In addition to content-based reductions, we also adopted the approach from DietCode [42]. We experimented with the random deletion method, which randomly deletes a certain proportion of tokens from the retrieved code snippets. The deletion rate was set to 30 percent. We also experimented with the attention weight deletion method. In DietCode, attention weight deletion is divided into two parts: deletion of entire lines and deletion of individual tokens. Instead of fully implementing both parts, we focused solely on the first step, deleting entire lines. This method categorizes lines based on keywords such as return, if, and while, calculates the attention weight of each type, then computes the attention weight of each line in detail, and ultimately deletes the lines with low attention weights.

4 EXPERIMENTAL DESIGN

4.1 Evaluation dataset

In this section, we choose RepoEval benchmark and select API invocation completion task to validate the effectiveness of our method. This task in RepoEval[41] comprises a total of 1600 test cases from 8 high-quality repositories. Each repository, created after February,1,2022 with MIT liscence or Apache V2.0 liscence, provides 200 inrepository defined APIs for completion. Each test case annotates repository's name and file path it belongs to, following by starting and ending of the vacancy and corresponding ground truth.

4.2 Baselines

In order to effectively assess the performance of RepoMinCoder method, we build up our baselines from two perspectives.

- **Non-iteration-based retrieval:** We utilize two non-iteration-based retrieval. Both retrieval split source code, encode and compare them by UniXcoder [9] and cosine similarity. The difference between two retrieval is whether the split procession is according by functions or by lines.
- **RepoCoder:** RepoMinCoder proposes a new strategy for selecting and sorting relevant repository context. As it is built on RepoCoder [41], we also choose RepoCoder as a baseline to showcase the improvement of our method.

4.3 Model selection

CodeGen [23, 25] is a generic term of a set of code generation models, containing multiple open-source published versions with various model sizes and different training data.

In our experiment, we choose CodeGen-mono 6B as a code completion model. CodeGen-mono 6B showcases excellent performance in HumanEval [3] and reaches 26.13% in pass@1 metric. This model is suitable for demonstrating the enhancement of our approach compared to other retrieval methods.

4.4 Hyper-parameters

In order to minimize the interference of hyperparameters, we keep the maximum tokens of prediction, the line length of the sliding window and the sliding size the same as RepoCoder choose, 100, 20, 10 respectively. However, we reduce the number of code snippets retrieved by RepoCoder [41] from 10 to 3 in order to combine code snippets retrieved by RepoMinCoder.

4.5 Evaluation metrics

Our research follows RepoCoder [41] and utilizes Exact Match (EM) and Edit Similarity (ES) to evaluate the performance of RepoMinCoder method. EM is a binary metric to indicate whether prediction completely matches ground truth. If true, the score is set to one, otherwise is zero. ES calculates Levinstein distance [16] to offer a more fine-grained perspective.

5 EXPERIMENTAL RESULTS

5.1 RQ1: What is the effectiveness of RepoMinCoder?

In the first set of experiments, we examine how our method performs in terms of performance. We show if our approach improves the code generation performance by comparing it to several methods that serve as baselines. This set of experiments is also used to demonstrate the amount of improvement.

According to Table 1, we have the following findings. When compared to RepoCoder over the same round of iterations, both the ES and EM of our method demonstrate superior performance. In comparison to the other two methods, ours outperform them in both the initial and final iterations. Furthermore, we also calculate the time taken for a single round in the two iterative methods. RepoCoder requires six hours and nineteen minutes, while our method takes seven hours and six minutes. Although a single round in our method consumes more time than RepoCoder, RepoCoder necessitates four rounds, whereas our method achieves nearly the same accuracy in only two rounds. Overall, our method significantly reduces the total time required.

Table 1: Overall performance comparison.

Method	Metric	RG Iterations			
		1	2	3	4
$RAG_{UniXcoder+function}$	EM	0.3281	-	-	-
	ES	0.6152	-	-	-
$RAG_{UniXcoder+line}$	EM	0.3725	-	-	-
	ES	0.6400	-	-	-
RepoCoder	EM	0.3669	0.3888	0.3913	0.3931
	ES	0.6420	0.6552	0.6553	0.6590
RepoMinCoder	EM	**0.3869**	**0.3994**	**0.4000**	-
	ES	**0.6499**	**0.6630**	**0.6610**	-

Let's delve deeper into the differences between the two iterations of RepoMinCoder. As shown in Table 2, there has been an improvement. However, upon closer examination of each library individually, we observe that not every library demonstrates performance improvement after the iteration. Iteration itself is not a very stable way to improve performance. After each iteration, incorrect content will be corrected, and correct content may become incorrect. However, in general, more errors are corrected with each iteration. If you are solely focused on improving performance, it is advisable to iterate until no further enhancements are observed. Nevertheless, if efficiency is also taken into account, continuous iteration may not be essential. The performance benefits derived from iterations are finite and variable, but the time required for each iteration is not insignificant.

One additional aspect that should be clarified in this method is the misaligned operation. In this scenario, the initial iteration aims to extract solely the preceding code snippet of the vacancy, which corresponds to the problem. Subsequently, we include a few lines from the extracted code snippet and remove the initial lines. This step is carried out to pinpoint the code that accurately matches the answer's location and to utilize it as a prompt. Through experimentation, we observe that this operation is beneficial and leads to a certain enhancement in performance.

It is important to emphasize that this operation is specific to this particular type of test set. The test set exclusively provides the code snippet above, omitting the subsequent code. In test sets where the in-context code is presented as a prompt, this operation is not feasible. Due to the answer being embedded within the question, dividing the code in such a manner is impracticable.

Summary: Our approach outperforms current state-of-the-art methods for repository-level code generation, improving the RepoEval benchmark by 3.3% in EM and 2.1% in ES.

5.2 RQ2:What is the impact of the number of code candidate snippets?

The MDL algorithm is highly sensitive to the number of candidate code snippets, and varying the quantity of candidates can produce different results. Therefore, we conduct an in-depth study on the

Table 2: Results on the first and the second iteration of RepoMinCoder method.

Repository in RepoEval	Metric	RG Iterations	
		1	2
huggingface_diffusers	EM	0.6550	0.6750
	ES	0.8107	0.8165
nerfstudio-project_nerfstudio	EM	0.2750	0.2550
	ES	0.5474	0.5578
awslabs_fortuna	EM	0.4600	0.4450
	ES	0.7328	0.7362
huggingface_evaluate	EM	0.3700	0.3900
	ES	0.6098	0.6258
alibaba_FederatedScope	EM	0.2500	0.2850
	ES	0.5860	0.6335
google_vizier	EM	0.3200	0.3650
	ES	0.6558	0.6824
pytorch_rl	EM	0.3250	0.3500
	ES	0.6010	0.6074
opendilab_ACE	EM	0.4400	0.4300
	ES	0.6553	0.6442

Table 3: Experimental results of the misaligned operation.

Method	EM	ES
with misaligned operation	0.3669	0.6420
without misaligned operation	0.3468	0.6222

Table 4: Experimental results of different number of candidate code snippets.

Number of candidate code snippets	EM	ES
10	0.3313	0.6076
5	0.3669	0.6400
3	0.3869	0.6499

number of candidate code snippets provided to the MDL algorithm. Based on our experiments, we find that the number of candidates has a significant impact on the final result.

First, according to the results in Table 4, we can see that when the number of candidates is 10, the EM is 0.3313 and the ES is 0.6076, which is less effective than when the MDL method is not used. When the number of candidates is reduced to 5, the EM increases to 0.3669 and the ES to 0.64, showing improvement compared to the previous case. When the number of candidates drops to 3, the EM further increases to 0.3869 and the ES to 0.6499, indicating additional improvement. Based on these data changes alone, it appears that an increase in the number of candidate snippets leads to a decrease in performance.

The common sense may suggest that more candidate snippets lead to better performance, but in fact the opposite is true. So, we try to find out the reasons. During the retrieval process, each code snippet is compared to the preceding code snippet in the vacancy for similarity to obtain a similarity score. We then select a certain number of code snippets with higher scores as candidates. According to the similarity comparison scores of candidate code snippets, we find that the more candidate code snippets there are in the test set, the greater the similarity gap between these snippets. Code snippets that are more similar are more likely to be useful. When the similarity gap widens, less similar snippets are not more valuable, even if they lose less information. In other words, from the perspective of similarity, the quality of too many candidate snippets cannot be effectively guaranteed. When the number of candidate snippets is reduced, the similarity gap between snippets is effectively controlled. As a result, not only is the efficiency improved, but also the time spent is also reduced.

According to this analysis, the MDL method can perform better when the quality of candidate code snippets is more consistent. However, this quality cannot be measured by similarity alone. Essentially, the quality we need to ensure is how much the candidate code snippet helps in code generation for the large language model. Similarity can reflect this quality to some extent, but dissimilarity is not necessarily useless. Relevance or other factors also need to be taken into account. In this experiment, we only compare by similarity and are limited by the length of the prompt. We demonstrate that more candidate code snippets do not always lead to better performance. However, we do not believe that fewer code snippets are necessarily better. It is likely best to select as many candidate snippets as possible while maintaining quality.

Summary: Through the above experiments, we explore the effect of the number of candidate code snippets on RepoMinCoder. It is not always the more the better. In order to balance performance and efficiency, we should provide a limited number of candidate code snippets while ensuring quality.

5.3 RQ3: how does the retrieval database's slicing method affect RepoMinCoder's performance?

In the repository-level code completion task, the repository needs to be sliced because the source code that can be used as a prompt is too long. We consider several slicing methods to determine whether different slicing methods affect performance. Taking both efficiency and performance into account, we aim to explore which method is better. We try to address these two questions.

For the study on the impact of slicing methods, we use UniXcoder [9] for vector generation and use cosine similarity uniformly to facilitate comparison, and we do not use the MDL method [37] for the second round of screening and sorting. We consider a total of four methods for comparison.

Function Granularity (FG): This method divides the code by function, ensuring the integrity of the code snippets to the greatest extent. However, the length of the snippets is very inconsistent, and a single snippet can even exceed the upper limit of the prompt length supported by the large language model.

Sliding Window (SW): This method uses a sliding window with a fixed number of lines to divide the code snippets. The window slides down a certain number of lines at a time, retrieving the content as a snippet each time. In our experiments, the window size is 20, and the sliding step is 10. This method effectively ensures that the length of each code snippet is relatively consistent and is

Table 5: Experimental results of different sliding method.

slicing method	EM	ES
FG	0.3281	0.6153
SW	0.3700	0.6421
FM	0.3263	0.6078
CM	0.3725	0.6400

suitable for both Jaccard [14] and cosine similarity. This method is also the one we used in RepoMinCoder.

Function Granularity with Maximum Length (FM): Similar to the first method, FM divides the code by function but limits the maximum length of a single snippet. If a snippet exceeds a certain limit, it is divided further. Short functions remain as one snippet, while long functions are divided into two or more snippets.

Combined Method (CM): This method combines the second and third methods. For all the code in the associated repository to be retrieved, the code snippets are divided using the sliding window method. For the preceding code snippet of the vacancy used as a query, the third method is employed. Starting from the vacancy, the code is divided upward. If the function signature is found or 20 lines are reached, the snippet is cut. If neither condition is met, continue upward.

According to Table 5, we find that among the four methods, the two methods using the sliding window show better performance, while the other two are not satisfactory. The EM of methods SW and CM is about 0.05 higher than that of methods FG and FM, and the ES is about 0.03 higher.

This is not what we expect at the beginning. Sliding by function granularity ensures code integrity as much as possible. In theory, this method should perform better, but in practice, the end result is not satisfactory. We try to figure out why. After observing the length of the split code snippets and the final selection, we formulate a hypothesis. The length of code snippets divided according to this method is extremely inconsistent, and some snippets are excessively long, even exceeding the upper limit of the prompt length. Such snippets never had a chance of being selected from the beginning. When the most similar snippet is too long, we settle for the next highest snippet. If this happens infrequently, the method is better overall. In practice, however, such forced delays occur more frequently than we imagine. As a result, we may seem to have a better choice, but we can't actually choose the better one. Despite this, the FM method also does not achieve good results. However, there is no denying that length does have an impact, and the CM method shows a slight improvement over the SW method.

We change our approach and start considering other possibilities. After comparing the differences between the two methods, we focus on sliding the window. When dividing snippets according to function granularity, we ensure that each line of code only appears in one snippet. When there is no line limit, dividing by function naturally ensures that the code at the boundary is not separated from the preceding or following text. However, when additional segmentation is done to control the length, we cannot actually guarantee this. In other words, the division method of FM method does not guarantee that every division is reasonable. The sliding window method also cannot guarantee this, or it even completely ignores this point. But sliding windows allow each line of code, especially the boundary code, to appear in more than one snippet. This allows the code at the boundary to be in different snippets with its above or below, respectively.

Although the sliding window introduces duplication, causing the same piece of code to appear in two or more snippets, it effectively ensures that various code snippets are taken into account. The code is a complete whole, and when we divide it, it is impossible to determine whether a line of code should be grouped with the preceding or following lines. The sliding window operation essentially includes both options. This allows us to compare and determine which grouping is better. However, it is possible that the same code will be selected multiple times. Therefore, the window size and the number of sliding lines need to be carefully considered. In this experiment, our window size is 20 lines, and the number of sliding lines is 10 lines.

Table 6: Experimental results of different prompt simplification methods.

Method	EM	ES	file size
in-file	0.2619	0.5645	-
RAG	0.3669	0.6420	16.6MB
only function signature	0.2488	0.5331	13.9MB
without annotations	0.368	0.6384	14.0MB
random deletion	0.2640	0.5550	14.0MB
delete according to attention weight	0.3200	0.6284	13.4MB

Summary: When dividing code snippets, we should pay attention to both code integrity and length. Prioritizing code integrity over length may result in suitable snippets exceeding the length limit and not being selected. On the other hand, focusing only on length can lead to prompts that are less helpful.

5.4 RQ4: How to improve the efficiency of the repository-level code completion process while maintaining performance?

Table 6 shows the results of the various reduction methods we tried, as well as the results of the methods used for comparison. Among them, "in-file" refers to the data generated by using only the preceding code of the vacancy without retrieval. "RAG" refers to data generated using the retrieval-augmented generation method to generate prompts. These two methods are used for comparison.

According to the results shown in Table 6, we find that performance is reduced to varying degrees after the deletion of code snippets. Among them, the method that only retains the function signature and the method that randomly deletes tokens show the greatest performance drops. We believe this is because both methods greatly undermine the integrity of the code snippets. When only function signatures are retained, the information provided to the large language model includes only the function name and the input and output content, lacking examples of general syntax. The function signature itself is primarily useful for generating function call statements. Moreover, function signatures provided as prompts may include class constructors and other special functions, which

offer limited value when only the signatures are given. Unfortunately, our examination of some search results revealed that such cases do occur and may not be rare. In contrast, the function call statements contained in the function body are more valuable as examples. This likely explains why retaining only function signatures drastically degrades performance.

The method of random deletion relies entirely on luck. While it has achieved some success in code summarization, it is clearly not suitable for code generation.

The method of cutting according to the attention weight has achieved some results. In this experiment, although EM decreased by 0.04 and ES decreased by 0.0116, the proportion is relatively small. More importantly, in this experiment, we do not train and set the attention weight for each type of sentence but directly used the existing set of attention weights. If more suitable parameters are found, the method will work better.

Deleting annotations have the best effect, with no drop in EM and less than a 0.01 drop in ES, while effectively deleting about 16% of the content. From a time perspective, this method effectively reduces the running time from the original six hours and 19 minutes to five hours and two minutes, only 80% of the original time. Our experimental results show that the removal of annotations brings a certain efficiency improvement. However, this method is still very crude and has some problems. First, we can't control how much content you delete, as it all depends on the amount of annotations in your code. This means that sometimes we may not be able to delete anything after a thorough retrieval, or we might almost completely delete an entire snippet. Second, annotations are deleted because we question the readability of their content. However, there is no denying that good annotations do exist. The removal of good comments can lead to performance degradation, even drastic degradation. Therefore, instead of just deleting annotations, it might be better to find ways to summarize and replace them.

Summary: These four methods have their own problems and have not achieved good results. Some methods are effective in reducing length but cause serious performance reduction. Other methods, although not causing significant performance reduction, the reduction is inconsistent. Our experiment might provide insights and experiences for future research.

6 DISCUSSION

6.1 Limitations

In this study, we explore the application of our method in the repository-level code completion task based on the large language model. Although this method achieve certain results, it still has many shortcomings, both inherent in the method itself and in the experiment, which require further research and exploration.

Firstly, our method is highly influenced by the quality of the candidate code snippets. The selection and sorting part of our method considers the loss caused by information transformation; reducing this loss can better leverage the role of the prompt. However, it all begins with ensuring that the prompts themselves are sufficiently useful. If the prompt does not contribute much to the problem, then even feeding it into a large language model will not help much. Therefore, the MDL method can perform better when the quality of the candidate code snippets is relatively consistent. Currently, the measurement of this quality remains to be further studied.

Second, we still need to consider the potential loss of efficiency caused by too many combinations of candidate code snippets. At present, however, due to the limitations of context length in most LLMs and the fact that individual code snippets are long, both the number of code snippets selected and the number of candidates are very limited. But there is no denying that with the advancement of technology, the prompt's length limits of the LLMs are becoming more and more relaxed. The Code Llama model [28] already supports context lengths up to 100K. In the future, we may still face the problem of too many combinations.

Finally, considering the method as a whole, although performance is improved by using our method. However, we employ an iterative approach to enhance performance. This approach significantly increases the cost in terms of time and space. It remains to be explored whether there is a feasible way to improve performance while reducing costs without the need for iteration.

6.2 Effect of prompt simplification

Current research on code generation focuses more on performance than on improving efficiency. In this study, we attempt to improve the efficiency of the repository code generation task based on LLMs by simplifying the prompts. Although the final experiment do not achieve very good results, it fails to improve efficiency while maintaining performance. However, our attempt provides some insights for related research and suggested some new directions. Future research may try to simplify the prompts by utilizing attention weights or by focusing on annotations.

7 CONCLUSION

In this study, we propose RepoMinCoder, an iterative retrieval-augmented code generation method. We introduce a screening and ranking step based on the perspective of information loss. While our research shows some initial results, the improvement brought by RepoMinCoder is still modest. In addition to the screening and ranking method, we also attempt to improve efficiency by reducing the content of the prompts. Although this aspect of the experiments do not yield significant improvements, it provides a novel idea and helps eliminate incorrect assumptions for other researchers.

In the future, it will be beneficial to enhance the retrieval method beyond similarity comparison by introducing more advanced context. Additionally, the issue of the excessive number of combinations in the method requires further improvement.

REFERENCES

[1] Toufique Ahmed and Prem Devanbu. 2023. Better Patching Using LLM Prompting, via Self-Consistency. *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2023), 1742–1746. https://api.semanticscholar.org/CorpusID:258999914

[2] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, C VageeshD, Arun Shankar Iyer, Suresh Parthasarathy, Sriram K. Rajamani, B. Ashok, and Shashank P. Shet. 2023. CodePlan: Repository-level Coding using LLMs and Planning. *ArXiv* abs/2309.12499 (2023). https://api.semanticscholar.org/CorpusID:262217135

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, and et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG]

[4] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2022. CoCoMIC: Code Completion By Jointly Modeling In-file and Cross-file Context. *ArXiv* abs/2212.10007 (2022). https://api.semanticscholar.org/CorpusID:254877371

[5] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, and Zhifang Sui. 2022. A Survey on In-context Learning. https://api.semanticscholar.org/CorpusID:255372865

[6] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. *ArXiv* abs/2002.08155 (2020). https://api.semanticscholar.org/CorpusID:211171605

[7] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida I. Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. InCoder: A Generative Model for Code Infilling and Synthesis. *ArXiv* abs/2204.05999 (2022). https://api.semanticscholar.org/CorpusID:248157108

[8] Peter D. Grünwald and Teemu Roos. 2019. Minimum Description Length Revisited. *ArXiv* abs/1908.08484 (2019). https://api.semanticscholar.org/CorpusID:201314867

[9] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Annual Meeting of the Association for Computational Linguistics*. https://api.semanticscholar.org/CorpusID:247315559

[10] Shivanshu Gupta, Sameer Singh, and Matt Gardner. 2023. Coverage-based Example Selection for In-Context Learning. *ArXiv* abs/2305.14907 (2023). https://api.semanticscholar.org/CorpusID:258865631

[11] Vincent J. Hellendoorn, Sebastian Proksch, Harald C. Gall, and Alberto Bacchelli. 2019. When Code Completion Fails: A Case Study on Real-World Completions. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 960–970. https://doi.org/10.1109/ICSE.2019.00101

[12] Samuel Humeau, Kurt Shuster, Marie-Anne Lachaux, and Jason Weston. 2019. Poly-encoders: Architectures and Pre-training Strategies for Fast and Accurate Multi-sentence Scoring. In *International Conference on Learning Representations*. https://api.semanticscholar.org/CorpusID:210063976

[13] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane A. Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2022. Few-shot Learning with Retrieval Augmented Language Models. *ArXiv* abs/2208.03299 (2022). https://api.semanticscholar.org/CorpusID:251371732

[14] P. Jaccard. 1912. THE DISTRIBUTION OF THE FLORA IN THE ALPINE ZONE.1. *New Phytologist* 11 (1912), 37–50. https://api.semanticscholar.org/CorpusID:85574559

[15] Xue Jiang, Yihong Dong, Lecheng Wang, Qiwei Shang, and Ge Li. 2023. Self-planning Code Generation with Large Language Model. *ArXiv* abs/2303.06689 (2023). https://api.semanticscholar.org/CorpusID:257495755

[16] Vladimir I. Levenshtein. 1965. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet physics. Doklady* 10 (1965), 707–710. https://api.semanticscholar.org/CorpusID:60827152

[17] Jia Li, Ge Li, Yongming Li, and Zhi Jin. 2023. Structured Chain-of-Thought Prompting for Code Generation. https://api.semanticscholar.org/CorpusID:258615421

[18] Jia Li, Yunfei Zhao, Yongming Li, Ge Li, and Zhi Jin. 2023. AceCoder: Utilizing Existing Code to Enhance Code Generation. https://api.semanticscholar.org/CorpusID:257901190

[19] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).

[20] Dianshu Liao, Shidong Pan, Xiaoyu Sun, Xiaoxue Ren, Qing Huang, Zhenchang Xing, Huan Jin, and Qinying Li. 2023. A3-CodGen: A Repository-Level Code Generation Framework for Code Reuse with Local-Aware, Global-Aware, and Third-Party-Library-Aware. https://api.semanticscholar.org/CorpusID:266162660

[21] Mingwei Liu, Tianyong Yang, Yiling Lou, Xueying Du, Ying Wang, and Xin Peng. 2023. CodeGen4Libs: A Two-Stage Approach for Library-Oriented Code Generation. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 434–445. https://doi.org/10.1109/ASE56229.2023.00159

[22] Zexiong Ma, Shengnan An, Bing Xie, and Zeqi Lin. 2024. Compositional API Recommendation for Library-Oriented Code Generation. *ArXiv* abs/2402.19431 (2024). https://api.semanticscholar.org/CorpusID:268091217

[23] Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. 2023. CodeGen2: Lessons for Training LLMs on Programming and Natural Languages. *ICLR* (2023).

[24] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Haiquan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *International Conference on Learning Representations*. https://api.semanticscholar.org/CorpusID:252668917

[25] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. *ICLR* (2023).

[26] Md. Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval Augmented Code Generation and Summarization. *ArXiv* abs/2108.11601 (2021). https://api.semanticscholar.org/CorpusID:237304122

[27] Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. In-Context Retrieval-Augmented Language Models. *Transactions of the Association for Computational Linguistics* 11 (2023), 1316–1331. https://api.semanticscholar.org/CorpusID:256459451

[28] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[29] Disha Shrivastava, H. Larochelle, and Daniel Tarlow. 2022. Repository-Level Prompt Generation for Large Language Models of Code. In *International Conference on Machine Learning*. https://api.semanticscholar.org/CorpusID:250072448

[30] Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. 2022. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239* (2022).

[31] J. Wang, Chengyu Wang, Chuanqi Tan, Jun Huang, and Ming Gao. 2023. Knowledgeable In-Context Tuning: Exploring and Exploiting Factual Knowledge for In-Context Learning. https://api.semanticscholar.org/CorpusID:262827483

[32] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. *ArXiv* abs/2109.00859 (2021). https://api.semanticscholar.org/CorpusID:237386541

[33] Jiayi Wei, Greg Durrett, and Işıl Dillig. 2023. Coeditor: Leveraging Contextual Changes for Multi-round Code Auto-editing. *ArXiv* abs/2305.18584 (2023). https://api.semanticscholar.org/CorpusID:258967769

[34] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Huai hsin Chi, F. Xia, Quoc Le, and Denny Zhou. 2022. Chain of Thought Prompting Elicits Reasoning in Large Language Models. *ArXiv* abs/2201.11903 (2022). https://api.semanticscholar.org/CorpusID:246411621

[35] Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari A. Sahraoui. 2023. Exploring Parameter-Efficient Fine-Tuning Techniques for Code Generation with Large Language Models. *ArXiv* abs/2308.10462 (2023). https://api.semanticscholar.org/CorpusID:261048999

[36] Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. 2024. Repoformer: Selective Retrieval for Repository-Level Code Completion. *ArXiv* abs/2403.10059 (2024). https://api.semanticscholar.org/CorpusID:268509840

[37] Zhiyong Wu, Yaoxiang Wang, Jiacheng Ye, and Lingpeng Kong. 2022. Self-Adaptive In-Context Learning: An Information Compression Perspective for In-Context Example Selection and Ordering. In *Annual Meeting of the Association for Computational Linguistics*. https://api.semanticscholar.org/CorpusID:254877590

[38] Daoguang Zan, B. Chen, Yongshun Gong, Junzhi Cao, Fengji Zhang, Bingchao Wu, Bei Guan, Yilong Yin, and Yongji Wang. 2023. Private-Library-Oriented Code Generation with Large Language Models. *ArXiv* abs/2307.15370 (2023). https://api.semanticscholar.org/CorpusID:260316265

[39] Daoguang Zan, Bei Chen, Zeqi Lin, Bei Guan, Yongji Wang, and Jian-Guang Lou. 2022. When Language Model Meets Private Library. In *Conference on Empirical Methods in Natural Language Processing*. https://api.semanticscholar.org/CorpusID:253237115

[40] Yury Zemlyanskiy, Michiel de Jong, Joshua Ainslie, Panupong Pasupat, Peter Shaw, Linlu Qiu, Sumit K. Sanghai, and Fei Sha. 2022. Generate-and-Retrieve: Use Your Predictions to Improve Retrieval for Semantic Parsing. In *International Conference on Computational Linguistics*. https://api.semanticscholar.org/CorpusID:252596262

[41] Fengji Zhang, B. Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Conference on Empirical Methods in Natural Language Processing*. https://api.semanticscholar.org/CorpusID:257663528

[42] Zhaowei Zhang, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. 2022. Diet code is healthy: simplifying programs for pre-trained models of code. *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (2022). https://api.semanticscholar.org/CorpusID:250113729

[43] Shuyan Zhou, Uri Alon, Frank F. Xu, Zhiruo Wang, Zhengbao Jiang, and Graham Neubig. 2022. DocPrompting: Generating Code by Retrieving the Docs. In *International Conference on Learning Representations*. https://api.semanticscholar.org/CorpusID:252734952